

PRODUCT REQUIREMENTS DOCUMENT (PRD)

Multi-Tenant SaaS Personal Budgeting Platform

Author: Senior Technical PM & Cloud Architect
Vercel

Status: MVP Ready

Target Stack: Vite, React, Supabase,

1. Problem Statement & Value Proposition

Problem Statement

Mengelola keuangan pribadi secara efektif tetap menjadi tantangan besar bagi mayoritas individu. Masalah utama terletak pada fragmentasi pencatatan, kurangnya visibilitas arus kas yang real-time, serta kerumitan antarmuka pada aplikasi finansial tradisional. Di sisi lain, isu privasi menjadi penentu utama: pengguna sangat sensitif terhadap data finansial mereka dan menuntut kepastian mutlak bahwa rincian pemasukan, pengeluaran, dan pola konsumsi mereka terisolasi secara aman serta tidak dapat diintip oleh pihak lain dalam platform yang sama.

Value Proposition

Platform SaaS Personal Budgeting ini hadir sebagai solusi MVP (Minimum Viable Product) yang menawarkan sistem pencatatan keuangan intuitif, cepat, dan memiliki arsitektur keamanan tingkat tinggi. Dengan mengadopsi model **Multi-Tenancy** yang diisolasi langsung pada level basis data menggunakan **Row Level Security (RLS) PostgreSQL via Supabase Cloud**, kita menjamin pemisahan data antar pengguna secara mutlak tanpa mengorbankan performa.

Aplikasi dibangun menggunakan kombinasi frontend **Vite + React.js** untuk antarmuka berbasis komponen yang responsif dan di-host pada platform **Vercel** dengan pipeline CI/CD otomatis. Solusi ini menghadirkan sistem finansial yang andal, efisien secara biaya infrastruktur, dan siap dikembangkan ke skala yang lebih besar.

Prinsip Akses Sistem Terisolasi

Sistem ini dirancang murni sebagai platform berbasis keanggotaan (authenticated application). Guna menjaga integritas RLS dan kepatuhan hukum privasi data finansial, platform tidak menyediakan mode "Guest" atau uji coba tanpa akun. Setiap interaksi data wajib melalui token otentikasi terenkripsi.

2. Minimum Viable Product (MVP) Features

Untuk memastikan akselerasi *time-to-market*, cakupan fitur MVP difokuskan secara ketat pada tiga fungsionalitas inti berikut:

2.1 Autentikasi & Registrasi Tenant

- **Registrasi Mandiri:** Pengguna dapat mendaftarkan akun baru menggunakan email dan kata sandi melalui layanan Supabase Auth.

- **Penyediaan Workspace Otomatis:** Setiap pendaftaran yang berhasil akan memicu pembuatan entitas profil baru yang bertindak sebagai *tenant* unik di dalam database, diidentifikasi melalui sandi enkripsi UUID.
- **Manajemen Sesi Aman:** Implementasi JSON Web Token (JWT) yang disimpan secara aman di sisi klien untuk memvalidasi setiap permintaan API menuju Supabase secara otomatis.

2.2 Manajemen Transaksi

- **Pencatatan Finansial (CRUD):** Pengguna dapat membuat (Create), membaca (Read), memperbarui (Update), dan menghapus (Delete) catatan keuangan mereka.
- **Atribut Transaksi:** Setiap transaksi wajib menyimpan informasi nominal (*amount*), jenis (pemasukan/pengeluaran), tanggal transaksi, deskripsi opsional, dan relasi kategori.
- **Kategori Kustom Per Tenant:** Pengguna dapat mengelola daftar kategori mereka sendiri (misal: Makanan, Transportasi, Belanja, Gaji) tanpa mengganggu pilihan kategori milik tenant lain.

2.3 Ringkasan Dashboard Finansial

- **Grafik Arus Kas Bulanan:** Visualisasi performa keuangan berupa perbandingan total akumulasi pemasukan dan pengeluaran pada bulan berjalan untuk memberikan pemahaman finansial yang cepat.
- **Indikator Batas Anggaran (Limit Tracker):** Komponen visual (*progress bar*) yang menunjukkan persentase total pengeluaran terhadap batas nominal anggaran bulanan yang telah ditentukan oleh pengguna di pengaturan profil mereka.

3. Multi-Tenant User Flow Diagram Deskriptif

Berikut adalah peta alur langkah-demi-langkah interaksi pengguna dengan penekanan pada mekanisme isolasi keamanan di balik layar:

- 1 **Pendaftaran Akun (Tenant Onboarding)**
Pengguna mengisi formulir registrasi di frontend React. Permintaan dikirim ke Supabase Auth. Setelah diverifikasi, Supabase menerbitkan UUID unik untuk pengguna tersebut dan menyisipkan baris baru di tabel penanda profil. Sesi JWT aktif diterbitkan ke browser pengguna.
- 2 **Inisialisasi Pengaturan Batas Anggaran**
Saat pertama kali masuk ke dashboard, pengguna diminta menentukan batas pengeluaran bulanan (*monthly budget*). Nilai ini disimpan ke dalam tabel *profiles* yang terikat langsung pada UUID pengguna.
- 3 **Pemuatan Data Dashboard (Isolated Fetching)**
Frontend React melakukan *query* data transaksi ke Supabase. Permintaan ini secara otomatis menyertakan JWT pengguna. Di level database, PostgreSQL membaca token tersebut, mengekstrak `auth.uid()`, dan mengaktifkan RLS sehingga database hanya mengembalikan baris transaksi yang memiliki nilai kolom `user_id` yang cocok. Data milik tenant lain dijamin tidak akan bocor.
- 4 **Pencatatan Transaksi Baru (Data Mutation)**
Pengguna menambahkan catatan pengeluaran baru. Frontend mengirimkan payload transaksi ke

Supabase. Kebijakan RLS memvalidasi operasi INSERT dengan memastikan bahwa kolom `user_id` yang dikirimkan sama dengan ID pengguna yang terenkripsi dalam JWT. Jika valid, data disimpan ke dalam tabel `transactions`.

5 Terminasi Sesi (Secure Logout)

Pengguna memilih opsi keluar dari aplikasi. Frontend React menghapus JWT dari penyimpanan lokal (session/local storage) dan memanggil fungsi `supabase.auth.signOut()`. Akses ke database terputus seketika, mencegah manipulasi data dari sesi lama.

4. Database Schema PostgreSQL for Supabase

Skema relasional ini menggunakan kolom `user_id` berjenis `uuid` di setiap tabel operasional sebagai kunci utama isolasi tenant (Tenant Identifier).

4.1 Tabel: `profiles`

Menyimpan data dasar pengguna dan preferensi anggaran global bulanan. Berelasi satu-ke-satu dengan tabel bawaan Supabase Auth (`auth.users`).

Nama Kolom	Tipe Data	Atribut Constraints	Keterangan / Deskripsi
<code>id</code>	<code>uuid</code>	PRIMARY KEY, REFERENCES <code>auth.users(id)</code>	ID Unik Pengguna (Tenant ID) dari Supabase Auth
<code>full_name</code>	<code>text</code>	NOT NULL	Nama lengkap pemilik akun
<code>monthly_budget</code>	<code>numeric(15,2)</code>	DEFAULT 0.00, CHECK (<code>monthly_budget >= 0</code>)	Batas nominal anggaran bulanan tenant
<code>created_at</code>	timestamp with time zone	NOT NULL, DEFAULT <code>now()</code>	Waktu penciptaan data profil

4.2 Tabel: `categories`

Menyimpan daftar kategori transaksi yang dikelola secara mandiri oleh masing-masing tenant.

Nama Kolom	Tipe Data	Atribut Constraints	Keterangan / Deskripsi
<code>id</code>	<code>uuid</code>	PRIMARY KEY, DEFAULT <code>gen_random_uuid()</code>	ID unik untuk setiap kategori

Nama Kolom	Tipe Data	Atribut Constraints	Keterangan / Deskripsi
user_id	uuid	NOT NULL, REFERENCES public.profiles(id)	Kunci Relasi Tenant pemilik kategori
name	text	NOT NULL	Nama Kategori (misal: 'Makanan', 'Gaji')
type	text	NOT NULL, CHECK (type IN ('income', 'expense'))	Penanda tipe: pemasukan atau pengeluaran
created_at	timestamp with time zone	NOT NULL, DEFAULT now()	Waktu pembuatan entri kategori

4.3 Tabel: transactions

Menyimpan data mutasi finansial harian pengguna.

Nama Kolom	Tipe Data	Atribut Constraints	Keterangan / Deskripsi
id	uuid	PRIMARY KEY, DEFAULT gen_random_uuid()	ID unik untuk setiap baris transaksi
user_id	uuid	NOT NULL, REFERENCES public.profiles(id)	Kunci Relasi Tenant pemilik transaksi
category_id	uuid	NOT NULL, REFERENCES public.categories(id)	Relasi ke tabel kategori transaksi
amount	numeric(15,2)	NOT NULL, CHECK (amount > 0)	Nilai nominal transaksi (harus positif)
transaction_date	timestamp with time zone	NOT NULL, DEFAULT now()	Tanggal dan waktu transaksi terjadi
description	text	NULL	Catatan atau deskripsi tambahan transaksi

5. Security & Row Level Security (RLS) Blueprint

Row Level Security (RLS) adalah barisan pertahanan utama pada arsitektur serverless Supabase. RLS memastikan aturan isolasi dieksekusi langsung di level mesin database PostgreSQL. Walaupun pihak luar

mengetahui endpoint API database kita, mereka tidak akan bisa melihat atau memanipulasi data tanpa token JWT yang sah dan sesuai.

5.1 Langkah Aktivasi RLS

Langkah pertama wajib mengaktifkan fitur RLS pada seluruh tabel operasional melalui SQL Editor Supabase:

```
-- Mengaktifkan pengamanan RLS pada tabel
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.categories ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.transactions ENABLE ROW LEVEL SECURITY;
```

5.2 Kebijakan Kebocoran Data (Anti-Data Leakage Policies)

Guna mengunci tabel transaksi (transactions) agar terisolasi penuh antar tenant, kita menerapkan empat aturan kebijakan (policy) berbasis fungsi otentikasi internal Supabase yaitu `auth.uid()`:

```
-- 1. Kebijakan Membaca Data (SELECT)
-- Tenant hanya diizinkan melihat data transaksi milik mereka sendiri
CREATE POLICY "Tenants can view their own transactions"
ON public.transactions FOR SELECT
USING (auth.uid() = user_id);

-- 2. Kebijakan Menambah Data (INSERT)
-- Tenant hanya dapat memasukkan data baru jika kolom user_id sesuai dengan ID otentikasi mereka
CREATE POLICY "Tenants can insert their own transactions"
ON public.transactions FOR INSERT
WITH CHECK (auth.uid() = user_id);

-- 3. Kebijakan Memperbarui Data (UPDATE)
-- Tenant hanya boleh memodifikasi transaksi miliknya sendiri
CREATE POLICY "Tenants can update their own transactions"
ON public.transactions FOR UPDATE
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id);

-- 4. Kebijakan Menghapus Data (DELETE)
-- Tenant hanya memiliki hak menghapus data transaksi yang sesuai dengan ID mereka
CREATE POLICY "Tenants can delete their own transactions"
ON public.transactions FOR DELETE
USING (auth.uid() = user_id);
```

Catatan Pengawasan Arsitektur

Pastikan kebijakan serupa diterapkan secara konsisten pada tabel `profiles` dan `categories`. Dengan blueprint ini, sistem memiliki jaminan keamanan berstandar tinggi (Zero-Trust Data Isolation) di mana kebocoran lintas data tenant dapat dicegah secara absolut secara native.